zadani

October 12, 2025

Vítejte u prvního projektu do SUI. V rámci projektu Vás čeká několik cvičení, v nichž budete doplňovat poměrně malé fragmenty kódu (místo je vyznačeno pomocí `None` nebo `pass`). Pokud se v buňce s kódem již něco nachází, využijte/neničte to. Buňky nerušte ani nepřidávejte. Snažte se programovat hezky, ale jediná skutečně aktivně zakázaná, vyhledávaná a – i opakovaně – postihovaná technika je cyklení přes data (ať už explicitním cyklem nebo v rámci `list`/`dict` comprehension), tomu se vyhýbejte jako čert kříží a řešte to pomocí vhodných operací lineární algebry.

Až budete s řešením hotovi, vyexportujte ho ("Download as") jako PDF i pythonovský skript a ty odevzdejte **pojmenované názvem týmu** (tj. loginem vedoucího). Dbejte, aby bylo v PDF všechno vidět (nezůstal kód za okrajem stránky apod.).

U všech cvičení je uveden orientační počet řádků řešení. Berte ho prosím opravdu jako orientační, pozornost mu věnujte, pouze pokud ho významně překračujete.

[22]:
```
import numpy as np
import copy
import matplotlib.pyplot as plt
import scipy.stats
```

1 Přípravné práce

Prvním úkolem v tomto projektu je načíst data, s nimiž budete pracovat. Vybudujte jednoduchou třídu, která se umí zkonstruovat z cesty k negativním a pozitivním příkladům, a bude poskytovat: - pozitivní a negativní příklady (`dataset.pos`, `dataset.neg` o rozměrech [N, 6]) - všechny příklady a odpovídající třídy (`dataset.xs` o rozměru [N, 6], `dataset.targets` o rozměru [N])

K načítání dat doporučujeme využít `np.loadtxt()`. Netrapte se se zapouzdřováním a gettery, berte třídu jako Plain Old Data.

Načtěte trénovací (`{positives,negatives}.trn`), validační (`{positives,negatives}.val`) a testovací (`{positives,negatives}.tst`) dataset, pojmenujte je po řadě `train_dataset`, `val_dataset` a `test_dataset`.

(6 řádků)

[23]:
```
class BinaryDataset:
    def __init__(self, positive : str, negative : str):
        self.pos = np.loadtxt(positive)
        self.neg = np.loadtxt(negative)
        self.xs = np.concatenate((self.pos, self.neg), axis=0)
```

```
        self.targets = np.concatenate((np.ones(len(self.pos)), np.
 ↪zeros(len(self.neg))))

train_dataset = BinaryDataset('positives.trn', 'negatives.trn')
val_dataset = BinaryDataset('positives.val', 'negatives.val')
test_dataset = BinaryDataset('positives.tst', 'negatives.tst')

print('positives', train_dataset.pos.shape)
print('negatives', train_dataset.neg.shape)
print('xs', train_dataset.xs.shape)
print('targets', train_dataset.targets.shape)
```

```
positives (2280, 6)
negatives (6841, 6)
xs (9121, 6)
targets (9121,)
```

V řadě následujících cvičení budete pracovat s jedním konkrétním příznakem. Naimplementujte proto funkci, která vykreslí histogram rozložení pozitivních a negativních příkladů z jedné sady. Nezapomeňte na legendu, ať je v grafu jasné, které jsou které.

Přiznak vyberte (pouze jeden z celkových šesti) takový, který bude nejlépe sloužit k účelům binární klasifikace. Příznak zvolte na základě podoby histogramu a stručně okomentujte důvod volby komentářem v kódu. Funkci zavoláte dvakrát, vykreslete histogram vybraného příznaku – pro trénovací a validační data

(5 řádků + komentář)

[24]:
```
FOI = 1  # Feature Of Interest

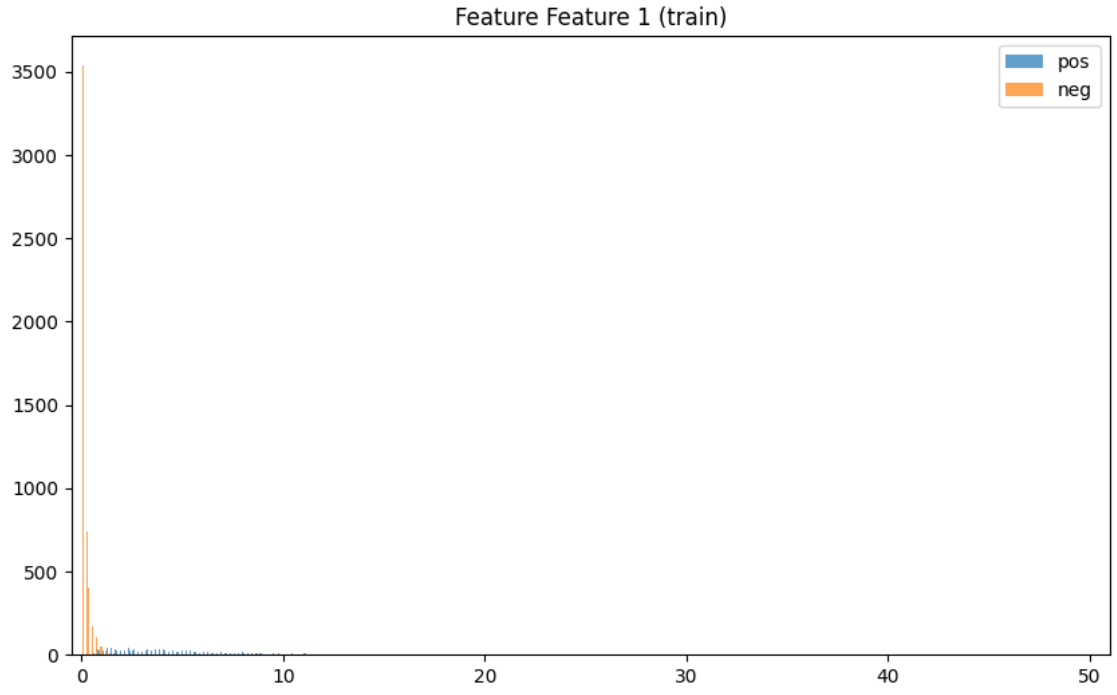
def plot_data(poss, negs, label):
    plt.figure(figsize=(10,6))
    plt.hist([poss, negs], bins='auto', alpha=0.7, label=['pos','neg'])
    plt.title(f'Feature {label}')
    plt.legend()
    plt.xlim(min(poss.min(), negs.min()) - 0.5, max(poss.max(), negs.max()) + 0.
 ↪5)
    plt.show()

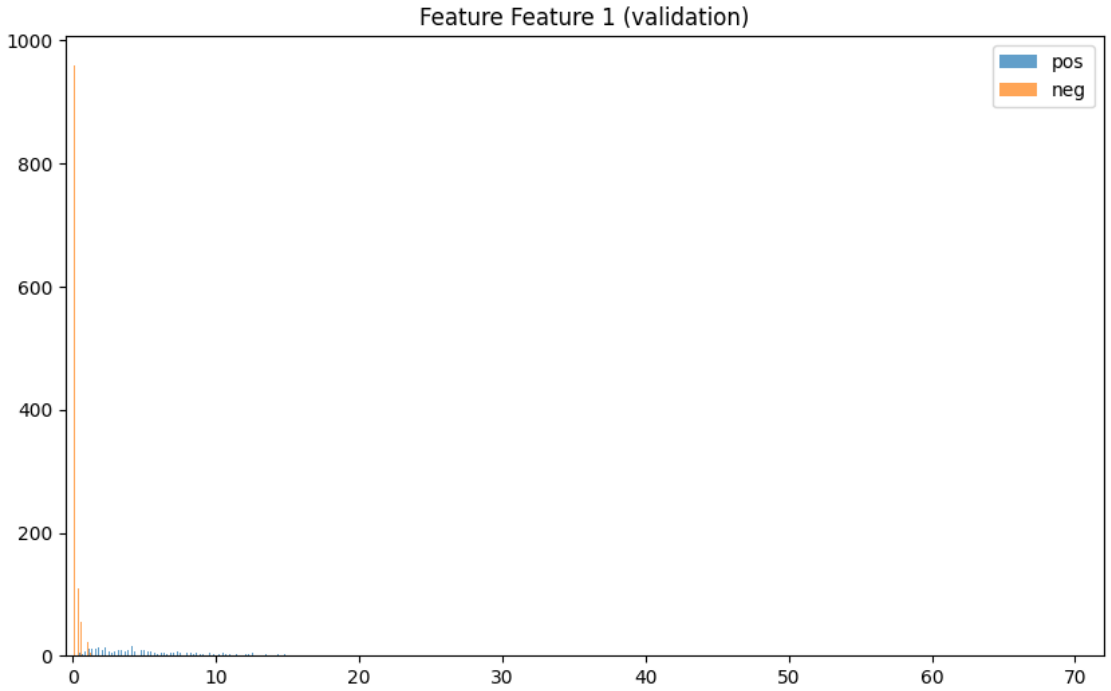

plot_data(train_dataset.pos[:, FOI], train_dataset.neg[:, FOI], f"Feature {FOI}␣
 ↪(train)")
plot_data(val_dataset.pos[:, FOI], val_dataset.neg[:, FOI], f"Feature {FOI}␣
 ↪(validation)")

"""
We were not provided with any description of the data whatsover.
Therefore it is naive to make assumptions about the data (prior probability).
```

```
For that reasons we find feature, where there is "least overlap", in the␣
 ↪certain bins.
This could also be calculated, however choosing purely based on visuals, we␣
 ↪choose the [1] feature, the data looks separable by 1d gaussians


Note: In "real" enviroment we always have to consider
how was your data sampled, if we do care about accuracy, false-positives,␣
 ↪false-negatives etc.
So in my opinion there is no "correct" answer given we know nothing about the␣
 ↪data.
"""
```





[24]:
```
'\nWe were not provided with any description of the data whatsover.\nTherefore
it is naive to make assumptions about the data (prior probability).\nFor that
reasons we find feature, where there is "least overlap", in the certain
bins.\nThis could also be calculated, however choosing purely based on visuals,
we choose the [1] feature, the data looks separable by 1d gaussians\n\n\nNote:
In "real" enviroment we always have to consider\nhow was your data sampled, if
we do care about accuracy, false-positives, false-negatives etc.\nSo in my
opinion there is no "correct" answer given we know nothing about the data.\n'
```

1.0.1 Evaluace klasifikátorů

Než přistoupíte k tvorbě jednotlivých klasifikátorů, vytvořte funkci pro jejich vyhodnocování. Nechť se jmenuje `evaluate` a přijímá po řadě klasifikátor, pole dat (o rozměrech [N, F]) a pole tříd ([N]). Jejím výstupem bude *přesnost* (accuracy), tzn. podíl správně klasifikovaných příkladů.

Předpokládejte, že klasifikátor poskytuje metodu `.prob_class_1(data)`, která vrací pole posteriorních pravděpodobností třídy 1 pro daná data. Evaluační funkce bude muset provést tvrdé prahování (na hodnotě 0.5) těchto pravděpodobností a srovnání získaných rozhodnutí s referenčními třídami. Využijte fakt, že `numpy`ovská pole lze mj. porovnávat se skalárem.

(3 řádky)

[25]:
```
def evaluate(classifier, inputs, targets):
    probs = classifier.prob_class_1(inputs)
    pred = (probs >= 0.5).astype(int)
    comb = (np.stack((pred,targets)))
```

```
    match = (comb[0,:] == comb[1,:]).astype(int)
    return np.sum(match) / len(targets)


class Dummy:
    def prob_class_1(self, xs):
        return np.asarray([0.2, 0.7, 0.7])

print(evaluate(Dummy(), None, np.asarray([0, 0, 1])))  # should be 0.66
```

```
0.6666666666666666
```

1.0.2 Baseline

Vytvořte klasifikátor, který ignoruje vstupní data a vrátí náhodnou predikci. Nezapomeňte, že jeho metoda `.prob_class_1(data)` musí vracet pole správné velikosti a podoby výstupu (třídy).

(4 řádky)

[26]:
```
class RandomClassifier:
    def __init__(self):
        np.random.seed(42)

    def prob_class_1(self, data):
        r =  np.random.rand(len(data))

        return (r >= 0.5).astype(int)

baseline = RandomClassifier()
val_acc = evaluate(baseline, val_dataset.xs[:, FOI], val_dataset.targets)
print('Baseline val acc:', val_acc)
```

```
Baseline val acc: 0.5025
```

2 Generativní klasifikátory

V této části vytvoříte dva generativní klasifikátory, oba založené na Gaussovu rozložení pravděpodobnosti.

Začněte implementací funce, která pro daná 1-D data vrátí Maximum Likelihood odhad střední hodnoty a směrodatné odchylky Gaussova rozložení, které data modeluje. Funkci využijte pro natrénovaní dvou modelů: pozitivních a negativních příkladů. Získané parametry – tzn. střední hodnoty a směrodatné odchylky – vypíšete.

(1 řádek)

[27]:
```
def mle_gauss_1d(data):
    mean = data.mean()
    var = data.var()
```

```
    return mean, np.sqrt(var)

mu_pos, std_pos = mle_gauss_1d(train_dataset.pos[:, FOI])
mu_neg, std_neg = mle_gauss_1d(train_dataset.neg[:, FOI])

print('Pos mean: {:.2f} std: {:.2f}'.format(mu_pos, std_pos))
print('Neg mean: {:.2f} std: {:.2f}'.format(mu_neg, std_neg))
```

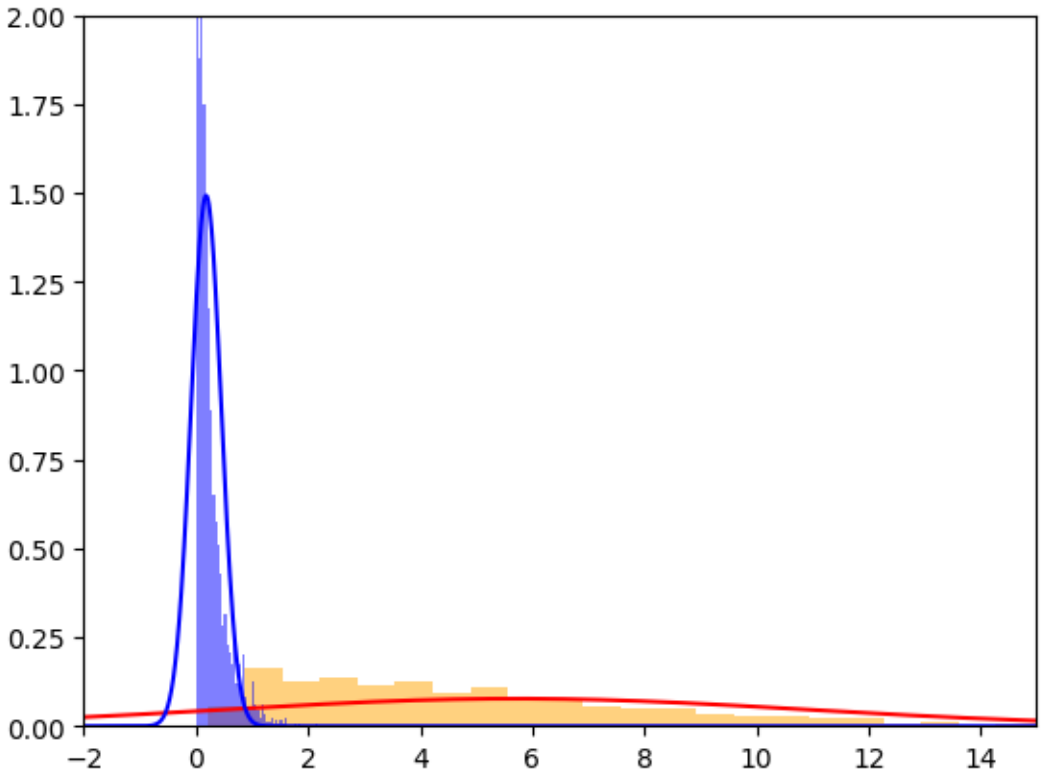
```
Pos mean: 5.71 std: 5.16
Neg mean: 0.18 std: 0.27
```

Ze získaných parametrů vytvořte `scipy`ovská gaussovská rozložení `scipy.stats.norm`. S využitím jejich metody `.pdf()` vytvořte graf, v němž srovnáte skutečné a modelové rozložení pozitivních a negativních příkladů. Rozsah x-ové osy volte dle vlastního uvážení (využijte `np.linspace`) a u volání `plt.hist()` nezapomeňte nastavit `density=True`, aby byl histogram normalizovaný a dal se srovnávat s modelem. Na grafu by mělo jít dobře vidět srovnání modelového rozložení se skutečnými daty.

(2 + 8 řádků)

[28]:
```
pos_g = scipy.stats.norm(loc=mu_pos, scale=std_pos)
neg_g = scipy.stats.norm(loc=mu_neg, scale=std_neg)

x = np.linspace(-2, 15, 1000)
plt.figure()
plt.hist(train_dataset.pos[:, FOI], bins=75, density=True, color='orange',␣
 ↪alpha=0.5)
plt.hist(train_dataset.neg[:, FOI], bins=75, density=True, color='blue',␣
 ↪alpha=0.5)
plt.plot(x, pos_g.pdf(x), color='red')
plt.plot(x, neg_g.pdf(x), color='blue')
plt.xlim(-2, 15)
plt.ylim(0,2)
plt.show()
```



Naimplementujte binární generativní klasifikátor. Při konstrukci přijímá dvě rozložení poskytující metodu `.pdf()` a odpovídající apriorní pravděpodobnost tříd. Dbejte, aby Vám uživatel nemohl zadat neplatné apriorní pravděpodobnosti. Jako všechny klasifikátory v tomto projektu poskytuje metodu `prob_class_1()`.

(9 řádků)

[29]:
```
class GenerativeClassifier2Class:
    def __init__(self, zero_dist, first_dist, prior_zero, prior_first):
        self.zero_dist = zero_dist
        self.first_dist = first_dist
        assert (prior_first + prior_zero) == 1
        self.prior_zero = prior_zero
        self.prior_first = prior_first

    def prob_class_1(self, data):
        l1 = self.first_dist.pdf(data)
        l0 = self.zero_dist.pdf(data)
        return np.divide(self.prior_first * l1, np.add(self.prior_zero * l0,␣
↪self.prior_first * l1))
```

Nainstancujte dva generativní klasifikátory: jeden s rovnoměrnými priory a jeden s apriorní pravděpodobností 0.75 pro třídu 0 (negativní příklady). Pomocí funkce `evaluate()` vyhodnotíte

jejich úspěšnost na validačních datech.

(2 řádky)

[30]:
```
classifier_flat_prior = GenerativeClassifier2Class(neg_g, pos_g, 0.5, 0.5)
classifier_full_prior = GenerativeClassifier2Class(neg_g, pos_g, 0.75, 0.25)

print('flat:', evaluate(classifier_flat_prior, val_dataset.xs[:, FOI],␣
 ↪val_dataset.targets))
print('full:', evaluate(classifier_full_prior, val_dataset.xs[:, FOI],␣
 ↪val_dataset.targets))
```

```
flat: 0.972
full: 0.976
```

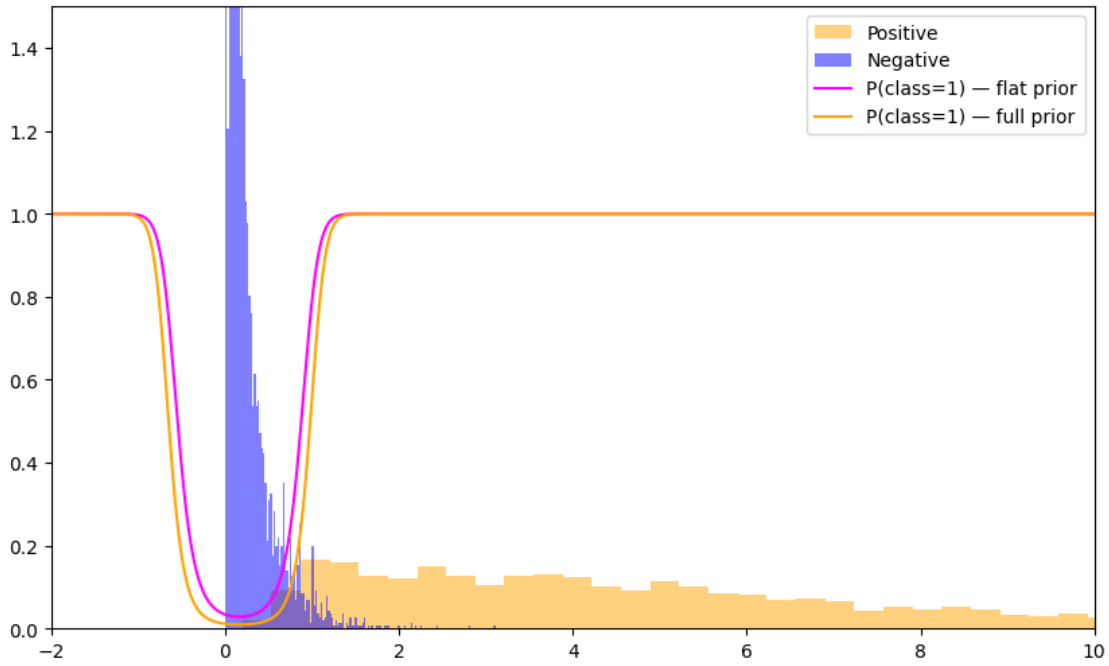
Vykreslete průběh posteriorní pravděpodobnosti třídy 1 jako funkci vámi zvoleného příznaku, opět ve vhodném rozsahu pro oba klasifikátory. Do grafu zakreslete i histogramy rozložení trénovacích dat, opět s `density=True` pro zachování dynamického rozsahu.

(8 řádků)

[31]:
```
x = np.linspace(-2,10,1000)

prio_flat = classifier_flat_prior.prob_class_1(x)
prio_full = classifier_full_prior.prob_class_1(x)

plt.figure(figsize=(10,6))
plt.hist(train_dataset.pos[:, FOI], bins=150, density=True, alpha=0.5,␣
 ↪color='orange', label='Positive')
plt.hist(train_dataset.neg[:, FOI], bins=150, density=True, alpha=0.5,␣
 ↪color='blue', label='Negative')
plt.plot(x, prio_flat, color='magenta', label='P(class=1) - flat prior')
plt.plot(x, prio_full, color='orange', label='P(class=1) - full prior')
plt.xlim(-2, 10)
plt.ylim(0,1.5)
plt.legend()
plt.show()
```



3 Diskriminativní klasifikátory

V následující části budete pomocí (lineární) logistické regrese přímo modelovat posteriorní pravděpodobnost třídy 1. Modely budou založeny čistě na NumPy, takže nemusíte instalovat nic dalšího. Nabitějších toolkitů se dočkáte ve třetím projektu.

[32]:
```
def logistic_sigmoid(x):
    return np.exp(-np.logaddexp(0, -x))

def binary_cross_entropy(probs, targets):
    return np.sum(-targets * np.log(probs) - (1-targets)*np.log(1-probs))

class LogisticRegressionNumpy:
    def __init__(self, dim):
        self.w = np.array([0.0] * dim)
        self.b = np.array([0.0])

    def prob_class_1(self, x):
        return logistic_sigmoid(x @ self.w + self.b)
```

Diskriminativní klasifikátor očekává, že dostane vstup ve tvaru `[N, F]`. Pro práci na jediném příznaku bude tedy zapotřebí vyřezávat příslušná data v správném formátu (`[N, 1]`). Doimplementujte třídu `FeatureCutter` tak, aby to zařizovalo volání její instance. Který příznak se použije, nechť je konfigurováno při konstrukci.

Může se Vám hodit `np.newaxis`.

(2 řádky)

[33]:
```
class FeatureCutter:
    def __init__(self, fea_id):
        self.fea_id = fea_id

    def __call__(self, x):
        return x[:, self.fea_id, np.newaxis]
```

Dalším krokem je implementovat funkci, která model vytvoří a natrénuje. Jejím výstupem bude (1) natrénovaný model, (2) průběh trénovací loss a (3) průběh validační přesnosti. Neuvažujte žádné minibatche, aktualizujte váhy vždy na celém trénovacím datasetu. Po každém kroku vyhodnoťte model na validačních datech. Jako model vracejte ten, který dosáhne nejlepší validační přesnosti. Jako loss použijte binární cross-entropii a logujte průměr na vzorek. Pro výpočet validační přesnosti využijte funkci `evaluate()`. Oba průběhy vracejte jako obyčejné seznamy.

(cca 11 řádků)

[34]:
```
def train_logistic_regression(nb_epochs, lr, in_dim, fea_preprocessor):
    model = LogisticRegressionNumpy(in_dim)
    best_model = copy.deepcopy(model)
    losses = []
    accuracies = []

    train_X = fea_preprocessor(train_dataset.xs)
    train_t = train_dataset.targets

    best_acc = 0.0
    for _ in range(nb_epochs):
        probs = model.prob_class_1(train_X)

        losses.append(binary_cross_entropy(probs, train_t) / len(train_t))

        grad = (probs - train_t) / len(train_t)
        grad_w = train_X.T @ grad
        grad_b = np.sum(grad)

        model.w -= lr * grad_w
        model.b -= lr * grad_b

        accuracies.append(acc := evaluate(
            model, fea_preprocessor(val_dataset.xs), val_dataset.targets
        ))

        if acc > best_acc:
            best_acc = acc
            best_model = copy.deepcopy(model)
```

```
    return best_model, losses, accuracies
```

Funkci zavolejte a natrénujte model. Uveďte zde parametry, které vám dají slušný výsledek. Pokuste se dostat na hodnotu přesnosti alespoň 0.8. Neměli byste potřebovat víc, než 100 epoch. Vykreslete průběh trénovací loss a validační přesnosti, osu x značte v epochách.

V druhém grafu vykreslete histogramy trénovacích dat a pravděpodobnost třídy 1 pro x, podobně jako výše u generativních klasifikátorů.

(1 + 5 + 8 řádků)

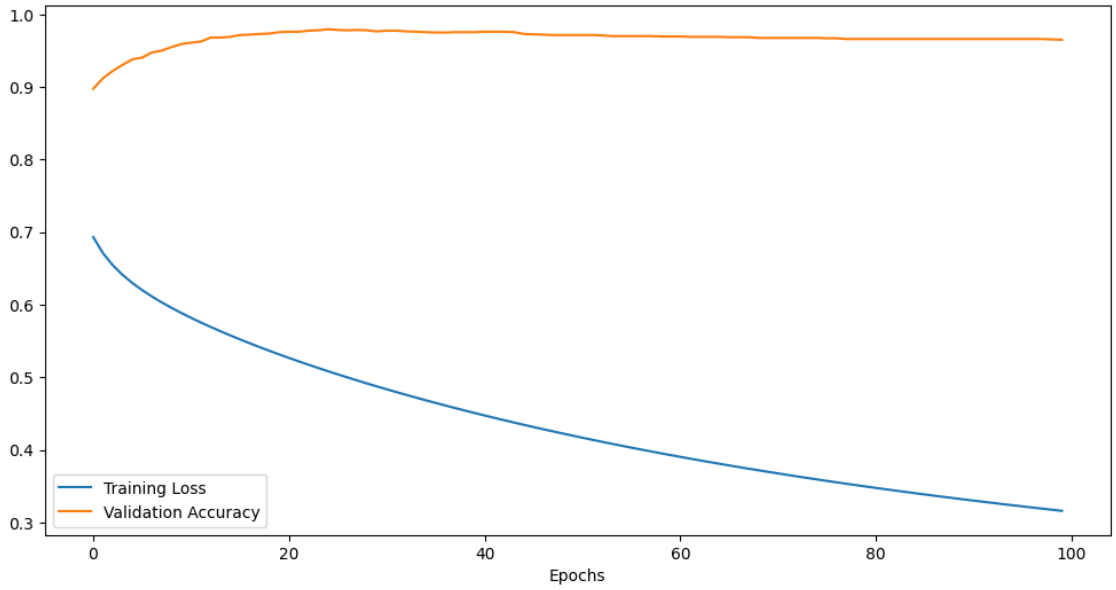
[35]:
```
disc_fea5, losses, accuracies = train_logistic_regression(
    nb_epochs=100,
    lr=0.05,
    in_dim=1,
    fea_preprocessor=FeatureCutter(FOI)
)
print('w', disc_fea5.w.item(), 'b', disc_fea5.b.item())
print('disc_fea5:', evaluate(disc_fea5, val_dataset.xs[:, FOI][:, np.newaxis],␣
 ↪val_dataset.targets))

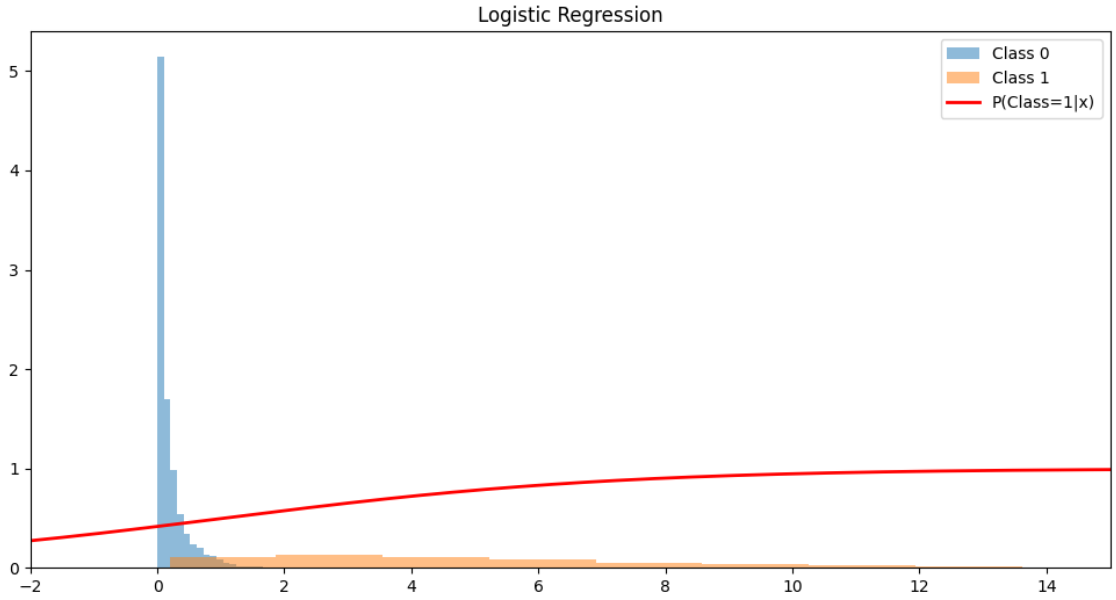
plt.figure(figsize=(12,6))
plt.plot(losses, label='Training Loss')
plt.plot(accuracies, label='Validation Accuracy')
plt.xlabel('Epochs')
plt.legend()
plt.show()

x_range = np.linspace(-2, 15, 200)[:, np.newaxis]
plt.figure(figsize=(12,6))
plt.hist(train_dataset.xs[train_dataset.targets == 0, FOI], bins=30, alpha=0.5,␣
 ↪label='Class 0', density=True)
plt.hist(train_dataset.xs[train_dataset.targets == 1, FOI], bins=30, alpha=0.5,␣
 ↪label='Class 1', density=True)

probs = disc_fea5.prob_class_1(x_range)
plt.plot(x_range, probs, 'r-', linewidth=2, label='P(Class=1|x)')
plt.legend()
plt.title('Logistic Regression')
plt.xlim(-2, 15)
plt.show()
```

```
w 0.3212428101969603 b -0.33768550641362505
disc_fea5: 0.9795
```





3.1 Všechny vstupní příznaky

V posledním cvičení natrénujete logistickou regresi, která využije všech šest vstupních příznaků. Zavolejte funkci z předchozího cvičení, opět vykreslete průběh trénovací loss a validační přesnosti. Měli byste se dostat nad 90 % přesnosti.

Může se Vám hodit `lambda` funkce.

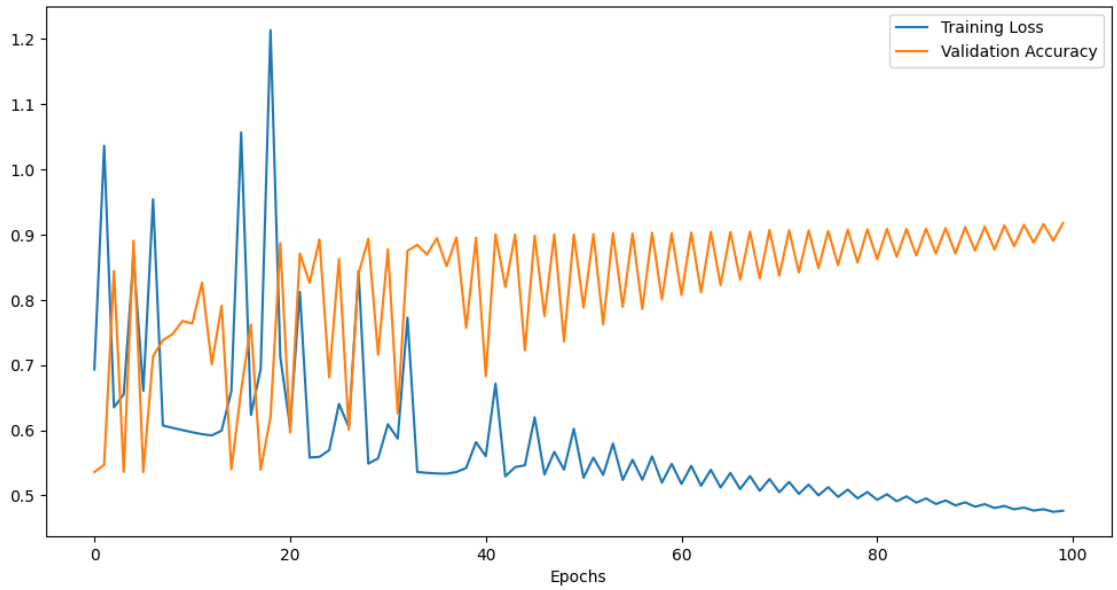
(1 + 5 řádků)

[36]:
```
disc_full_fea, losses, accuracies = train_logistic_regression(
    nb_epochs=100,
    lr=0.0096,
    in_dim=6,
    fea_preprocessor=lambda x: x
)

print('w', disc_full_fea.w, 'b', disc_full_fea.b)
print('disc_full_fea:', evaluate(disc_full_fea, val_dataset.xs, val_dataset.
 ↪targets))

plt.figure(figsize=(12,6))
plt.plot(losses, label='Training Loss')
plt.plot(accuracies, label='Validation Accuracy')
plt.xlabel('Epochs')
plt.legend()
plt.show()
```

```
w [ 0.01659564  0.1314698  -0.03100236 -0.2132017   0.14373328 -0.11220214] b
[-0.26679353]
disc_full_fea: 0.9175
```



4 Závěrem

Konečně vyhodnoťte všech pět vytvořených klasifikátorů na testovacích datech. Stačí doplnit jejich názvy a předat jim odpovídající příznaky. Nezapomeňte, že u logistické regrese musíte zopakovat formátovací krok z `FeatureCutteru`.

[37]:
```
xs_full = test_dataset.xs
xs_foi = test_dataset.xs[:, FOI]
targets = test_dataset.targets

print('Baseline:', evaluate(baseline, xs_full, targets))
print('Generative classifier (w/o prior):', evaluate(classifier_flat_prior,␣
 ↪xs_foi, targets))
print('Generative classifier (correct):', evaluate(classifier_full_prior,␣
 ↪xs_foi, targets))
print('Logistic regression:', evaluate(disc_fea5, xs_foi[:, np.newaxis],␣
 ↪targets))
print('logistic regression all features:', evaluate(disc_full_fea, xs_full,␣
 ↪targets))
```

```
Baseline: 0.504
Generative classifier (w/o prior): 0.9615
Generative classifier (correct): 0.964
Logistic regression: 0.9645
logistic regression all features: 0.9195
```

Blahopřejeme ke zvládnutí projektu! Nezapomeňte (1) spustit celý notebook načisto (Kernel -> Restart & Run all), (2) zkontrolovat, že všechny výpočty prošly podle očekávání, a (3) před odevzdáním pojmenovat soubory loginem vedoucího týmu.

Mimochodem, vstupní data nejsou synteticky generovaná. Nasbírali jsme je z baseline řešení historicky prvního SUI projektu; vaše klasifikátory v tomto projektu predikují, že daný hráč vyhraje dicewars, takže by se daly použít jako heuristika pro ohodnocování listových uzlů ve stavovém prostoru hry.

[37]: